



Project A: An Optical Ray Tracer

Introduction

Optical ray tracing is a powerful technique that is at the heart of most optical design packages. In this project you will design and code your own optical ray tracer and use it to investigate quantitatively some optical systems.

Project Aims

- learn how to use GitHub via the use of GitHub Classroom in VS code
- learn to use unit tests to test and debug your code
- design and write an optical ray tracer in Python using object-oriented programming
- test and verify the operation of the ray-tracer
- use the ray-tracer to investigate the imaging performance of simple lenses



Task 1.

If you would like to choose this project then please accept the GitHub assignment by clicking [here](#).

Section 1: Designing an Optical Ray Tracer

In a sequential optical ray tracer, the optical system is modelled by propagating rays through the optical system using the principles of geometrical optics. When the ray encounters an interface between two dielectric media in the system, such as the surface of a lens, the ray is refracted (using Snell's law). Propagation continues to the next surface and so on until the ray reaches the output of the system. It is possible to include different types of optical surfaces including reflecting surfaces.

You will build a simple 3-D optical ray tracer and use it to model the behaviour of some simple optical systems. We will use an object-oriented approach to designing the model. A ray can be represented by a series of points that describe the path through the optical system and a direction of propagation.

In our model, an optical surface object will have a number of properties including refractive indices, and geometrical properties of surface shape. It will also have methods for determining where rays intercept with it and for calculating the effect of the surface on a ray, such as refraction or reflection.

Optical Rays

We shall begin by designing a class to represent an optical ray. The ray will be represented by a list of points and a normalised direction (each point as well as the normalised direction should be 3D vectors in cartesian coordinates).

Task 2.

Create a new module `rays.py` which will store our `Ray` class. Add some documentation to your module so it's clear what it does. Now create a new class in the module called `Ray` that can be used to represent an optical ray. You should be able to initialise your Ray object as below:

```
>>> ray1 = Ray()
>>> ray2 = Ray(pos=[0, 0, 0], direc=[0, 0, 1])
```

Taking the z-axis as the optical axis, your Ray should have a default position of `[0, 0, 0]` and a default direction of `[0,0,1]`. Internally, you should store these as NumPy arrays. Remember to name mangle your attributes and add access methods for these variables. Since your ray will be propagated through the system, it should have a method to add a new point and direction (you should store the collection of points internally as a list but this is not necessary for the direction). It should also have a method that returns all the points along the ray. Your class should at least have the following methods:

`class Ray`

`pos()`

return the current point of the ray

`direc()`

return the current ray direction

`append(pos, direc)`

append a new point and direction to the ray

`vertices()`

return all the points along the ray in the form of a list containing the numpy array position vectors.

Test the behaviour of your class. Can you make instances of it and initialize it? Check that the method functions work sensibly and you can add new points to the ray which are recorded properly by the `vertices` method. Make sure you include appropriate documentation for your class.

Optical Elements

We shall represent our optical system using optical elements such as refracting surfaces, output plane etc., which will be stored in an `elements.py` module. In our sequential ray

tracer, a ray will be propagated through each optical element in turn. The object representing the optical element will be responsible for propagating the ray through it. We shall begin with a general base class, `OpticalElement`. We will not be directly instantiating this class but rather using it as a base for our different derived classes.

Task 3.

Create a new module called `elements.py`. Inside this new module, copy the following code to create a base class for our optical elements.

```
class OpticalElement:
    def intercept(self, ray):
        raise NotImplementedError('intercept() needs to be implemented in
derived classes')
    def propagate_ray(self, ray):
        raise NotImplementedError('propagate_ray() needs to be implemented
in derived classes')
```

Even though we won't be implementing this class directly, it's important that you understand what these methods are for. To this end we provide the class documentation below. Please read through it and make sure it's clear.

`class OpticalElement`

`intercept(self, ray)`

This method takes a `Ray` object as an argument and calculates where its path will intercept with the element. If no intercept is found it should return **None**

`propagate_ray(self, ray)`

This method takes a `Ray` object as an argument and will propagate it through the element. To do this, the method will need to:

1. Work out if and where the ray's path will intercept with the object.
2. Calculate any change that the element may cause to the ray's direction vector.
3. Update the ray's internal state to reflect this new intercept position and direction.

Then for a `Ray` object, and a list of different optical elements (all of which will have a `propagate_ray` method) one might propagate the ray through the system thusly:

```
>>> for elem in elements:
...     elem.propagate_ray(ray)
```

Refracting surfaces

Most optical surfaces are spherical (or planar). We will now design a class to represent a spherical refracting surface. We shall initially restrict ourselves to spherical surfaces centred on the optical axis (which by convention we shall take to be the z -axis). Then a spherical refracting surface can be represented by five parameters:

- **z_0** - the intercept of the surface with the z -axis
- **curvature** - the curvature of the surface (a signed quantity with magnitude $1/(\text{radius of curvature})$)
- **n_1 , n_2** - the refractive indices either side of the surface
- **aperture** (*aperture radius*) - the maximum extent of the surface from the optical axis

_images/refracting_surface.png

We shall use the convention that a positive curvature corresponds to the centre of curvature at $z > z_0$ (as in the image above) and negative curvature corresponds to a centre of curvature at $z < z_0$. A curvature of zero corresponds to a plane surface (i.e., infinite radius of curvature.)

Task 4

In the `elements.py` module, create a new class `SphericalRefraction` (inheriting from `OpticalElement`) that you will use to represent a spherical refracting surface. It should be able to be initialized with the five parameters described above. If done correctly, you should be able to initialise your class as below:

```
>>> sr = SphericalRefraction(z_0=10, aperture=5, curvature=0.2, n_1=1., n_2=1.5)
```

In addition, you should provide accessor methods to access the attributes as below:

```
class SphericalRefraction
```

```
    z_0(self)
```

```
        return the intercept of the surface with the z-axis
```

```
    aperture(self)
```

```
        return the aperture (maximum extent of the surface from the optical axis)
```

```
    curvature(self)
```

```
        return the curvature of the surface
```

```
    n_1(self)
```

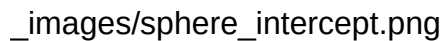
```
        return the refractive index on the left ( $z < z_0$ ) of the surface
```

`n_2(self)`

return the refractive index on the right ($z > z_0$) of the surface

Intersection of a ray with a spherical surface

Consider a ray starting at a point $\mathbf{P}$ which intersects a sphere of radius R centred on $\mathbf{O}$ at a point $\mathbf{Q}$. $\mathbf{r}$ is the vector from the sphere's centre $\mathbf{O}$ to the ray's starting point $\mathbf{P}$ and $\hat{\mathbf{k}}$ is the unit vector in the direction of the ray as depicted below.

_images/sphere_intercept.png

Then the distance l from $\mathbf{P}$ to $\mathbf{Q}$ can be found by solving the quadratic equation:

$$\mathbf{r} + l\hat{\mathbf{k}}^2 = R^2 \quad (1)$$

which gives

$$l = -\mathbf{r} \cdot \hat{\mathbf{k}} \pm \sqrt{(\mathbf{r} \cdot \hat{\mathbf{k}})^2 - (|\mathbf{r}|^2 - R^2)} \quad (2)$$

Task 5

Since our new class `SphericalRefraction` is an `OpticalElement` we will need to implement an `intersect(ray)` method within it.

```
class SphericalRefraction
```

```
...
```

```
    intersect(self, ray)
```

This method should calculate the first valid intercept of a ray with the spherical surface. If there is no valid intercept, then your method should return **None**.

Note

Remember a line can have two intercepts with a sphere — make sure you get the correct intercept for your surface. This should be the first intercept along the forward direction of the ray that lies within the aperture of the surface.

Refraction at a surface

Once a ray is intercepted by a surface it is refracted according to Snell's law.

_images/snell.png

$$n_1 \sin \theta_1 = n_2 \sin \theta_2 \quad (3)$$

where n_1 and n_2 are the refractive indices either side of the surface and θ_1 and θ_2 are the angles of the rays with respect to the surface normal $\hat{\mathbf{n}}$.

Task 6

Since Snell's law is a generic physics principle and not related specifically to our elements, we shall create a new module for it called `physics.py`.

- Create a new module called `physics.py`.
- Write a function to implement Snell's law refraction in the new `physics.py` module. Your function should be called `refract()` and should take an incident direction, a surface normal, and the refractive indices n_1 and n_2 as parameters thusly.

`refract(direc, normal, n_1, n_2)`

This function should return the refracted ray direction (as a normalised numpy array).

If the ray is subject to total internal reflection (i.e., $\sin \theta_1 > n_2/n_1$) this function should return **None**.

Now that we can find the intercept of a ray with our surface as well as refracting the ray, we can finish off implementing the functionality of our class by adding the necessary

`propagate_ray(ray)` method.

Task 7

In your `SphericalRefraction` class, we will also need to implement the method `propagate_ray(ray)` described in the base class `above`.

`class SphericalRefraction`

...

`propagate_ray(self, ray)`

propagate the `Ray` object through this surface, appending the new position and direction to the ray's internal state.

Note

A new position and direction should **not** be appended if there is no valid intercept **or** total internal reflection occurs.

With this implemented, we should now have the basics necessary to trace a ray. All your investigation code should be stored in the module `analysis.py` which you have already been provided with. You should not change the structure of the functions (i.e. their names or parameters) in this file, instead simply implement the functions as necessary.

Task 8

To test your code, we will be implementing the `task8()` function contained in `analysis.py` module. Write some test code to make sure that your `propagate_ray` method (and hence your `intercept`, `append` and `refract()` functions) behaves as you expect by propagating a range of initial rays through your surface and checking that their positions and directions are correctly updated.

Output plane

The final element in your sequence of optical elements should be the `OutputPlane`. As with our `SphericalRefraction` class, rays should be able to be propagated to this element but their direction should remain unchanged as they terminate at this point and do not undergo refraction.

Task 9

Back in your `elements.py` module, write a class `OutputPlane` (that is an `OpticalElement`). You should be able to construct it with a single argument `z_0` as below.

```
>>> op = OutputPlane(z_0=250.)
```

Properly implement the necessary methods `intercept()` and `propagate_ray()`.

Section 2: Raytracing Investigations

Testing your Model

A single spherical refracting surface (with media of different refractive index on either side) is the simplest system that can form an image of an object.

An incident plane wave, corresponding to light from a point object at infinity, can be represented by parallel input rays. In an ideal imaging system, all such rays would meet at the output focal plane. In practice this does not happen because of spherical aberration — rays deviate from the ideal (paraxial) behaviour as the distance of the input ray from the optical axis increases.

Let's try modelling this...

Task 10.

Implementing `task10()` in your `analysis.py` module, start with a simple system comprising a single spherical refracting surface and an output plane. Consider a spherical surface at $z_0 = 100\text{mm}$ with curvature 0.03 mm^{-1} , aperture 34mm and refractive indices $n_1 = 1.0$ and $n_2 = 1.5$ and an output plane at $z_0 = 250\text{ mm}$.

Note

If not otherwise specified you should continue using these `SphericalRefraction` parameters for the remaining tasks until Task15.

Try tracing the following rays from different points in the input plane ($z = 0\text{ mm}$) to the output plane.

Initial Ray Positions:

- `[0, 4, 0]`
- `[0, 1, 0]`
- `[0, 0.2, 0]`
- `[0, 0, 0]`
- `[0, -0.2, 0]`
- `[0, -1, 0]`
- `[0, -4, 0]`

When you have traced rays through the optical system, you should plot their paths using `matplotlib()` and return the figure object.

Check you can see signs of spherical aberration?

The *paraxial focus* is the ideal focus in the limit of a narrow input beam. To determine the paraxial focus of this particular surface, we will need to trace some *paraxial rays* (close to the optical axis) to the `OutputPlane` and see where they cross the optical axis.

Task 11

Before tracing these rays, let's first add some additional functionality. Add a new method to your `SphericalRefraction` class called `focal_point` which should return the calculated focal point of the surface.

The new method should have the following signature.

```
class SphericalRefraction
```

```
...
```

focal_point(self)

This method should calculate and return the focal_point for the surface.

Now, in `task11()` of your `analysis.py` module, trace paraxial rays with the initial positions given below to the OutputPlane (placed at the focal point defined above).

- [0.1, 0.1, 0]
- [0, 0, 0]
- [-0.1, -0.1, 0]

Your `task11()` function should also make a plot of the ray paths and return both the figure object as well as the calculated focal point in a tuple.

How does the focal point you see from your simulation (the plot) compare to the calculated value?

Now we have shown that your ray tracing code is working and the focal point matches what we can calculate, let's make it easier to use. At the moment you have to manually create each `Ray` and manually trace them through each `OpticalElement`. This is tedious and we can do better. We shall create code to represent a bundle of `Ray` objects and allow them to propagate themselves through each element.

FOER Task 12

Create a `RayBundle` class in the `rays.py` module that will represent a circular (transverse to the optical axis) bundle of `Ray` objects. You should initialise the rays in circular rings by utilising the polar generation code in the worksheets. You should be able to construct your bundle as below:

```
>>> rb = RayBundle()
>>> rb = RayBundle(rmax=5., nrings=5., multi=6)
```

Take your default values to be $r_{max} = 5\text{mm}$, $n_{rings} = 5$ and $multi = 6$

FOER Note

Unless stated otherwise you should use these values as default for this and future tasks.

We now need to take care of propagation of these Rays as well as adding a convenience plotting method.

- Create a method `propagate_bundle()` to the `RayBundle` class that will take a list of

`OpticalElement` objects and sequentially propagate each ray through them.

- Create a method `track_plot()` in your `RayBundle` class that will create a plot of the path of each of the rays in the bundle.

The signatures for the new methods are given below:

class `RayBundle`

`propagate_bundle(self, elements)`

This method should, given a list of `OpticalElement` objects, propagate each ray in the bundle through each element.

`track_plot(self)`

This method should create a plot of the path of each ray in the bundle, returning the matplotlib figure object.

- Implement the `task12()` function in the `analysis.py` module to create a ray bundle and propagate it through the default surface (defined above) up to an `OutputPlane` (located at the `focal_point`). The function should then plot the tracks of the rays in the ray bundle. Ensure that the `task12()` function returns the matplotlib figure.

An estimate of the size of the geometrical focus is given by the RMS (spread from the optical axis) of the rays in the bundle. A **spot plot** is a diagram (in the plane transverse to the optical axis) showing the rays x-y positions at a certain **z** position.

Task 13

- Create a new method `rms()` for your `RayBundle` class that will calculate the RMS size of the bundle.
- Create another method `spot_plot()` that will plot the spot diagram for your ray bundle.

The signature for the new methods is below:

class `RayBundle`

...

`rms(self)`

This method calculates and returns the RMS size of the bundle

`spot_plot(self)`

This method should plot the x-y position of each ray in the bundle at the rays'

current positions. IT should return the matplotlib figure object.

- Implement the `task13()` function in `analysis.py` to create a RayBundle and propagate it through the default SphericalRefraction surface up to the OutputPlane (located at the focal point). It should then plot the spot plot and return both the matplotlib figure object and the calculated rms at the focal point.

Until this point we have only been investigating geometrical optics, which ignores the effects of diffraction. The diffraction scale of an optical system is:

$$\Delta x = \lambda f / D \quad (4)$$

where λ is the wavelength, f the focal length and D the aperture diameter or diameter of the input beam in this case.

Task 14

In `task14()`, you should investigate how the RMS value and the diffraction scale varies with bundle radius for a wavelength of $\lambda = 588 \text{ nm}$.

Take ray bundles of radii between 0.1mm and 10mm and propage them through the default SphericalRefraction surface up to the OutputPlane(located at the focal point). Plot the radial dependence of both the RMS value and the diffraction scale on the same axis.

Your function should return this graph (matplotlib figure object) along with both the RMS value and diffraction scale value for input beam radius 2.5mm as a tuple.

Consider what this means about the accuracy of your simulation for different bundle sizes.

The Plano-Convex Lens

One of the simplest lenses to investigate is the plano-convex lens. It has two optical surfaces: one planar, one convex. The behaviour of rays travelling through this plano-convex lens will vary depending on the orientation of the lens (i.e. planar side or convex side first). We shall now investigate these differences.

Task 15

Start by creating a new module `lenses.py` in which we will create a new class `PlanoConvex` to represent this lens.

Implement the `PlanoConvex` class such that you can construct it with the following

parameters:

- **z_0** - the intercept of the left most surface with the z-axis
- **curvature1** & **curvature2** - the curvatures of the left and right surface (**Note** depending on your exact implementation you may only need one of these, in which case it should be simply **curvature**)
- **n_inside** & **n_outside** - the refractive indices both inside and outside the lens.
- **thickness** - the thickness of the lens on the optical (z) axis
- **aperture** - The maximum extent of the surface from the optical (z) axis.

If done correctly, you should be able to instantiate your classes in one of the ways below:

```
>>> pc = PlanoConvex(z_0=100, curvature1=0., curvature2=-0.02, n_inside=1.5168,
n_outside=1., thickness=5., aperture=50.)
>>> cp = PlanoConvex(z_0=100, curvature1=0.02, curvature2=0., n_inside=1.5168,
n_outside=1., thickness=5., aperture=50.)

or

>>> pc = PlanoConvex(z_0=100, curvature=-0.02, n_inside=1.5168, n_outside=1.,
thickness=5., aperture=50.)
>>> cp = PlanoConvex(z_0=100, curvature=0.02, n_inside=1.5168, n_outside=1.,
thickness=5., aperture=50.)
```

Consider a plano-convex lens which has a plane surface located at $z_0 = 100\text{mm}$, (you can model this with a `SphericalRefraction` object with zero curvature **or otherwise**) and a concave surface with a curvature of -0.02 mm . The separation of the surfaces (thickness) on the optical axis is 5 mm and the refractive index of the glass is $n_{\text{glass}} = 1.5168$ (assume $n_{\text{air}} = 1$) at a wavelength of $\lambda = 588\text{ nm}$. By placing the `OutputPlane` at the focal point, we will investigate the performance of the lens for focusing collimated light with a beam radius of 5 mm .

- Model the system first with the planar surface facing the input, then with the convex surface facing the input.

For each orientation, you should propagate a ray bundle through the lens system, to the `OutputPlane`, and create a spot plot at their focal point (you will need to find out what this is, remembering that it will be different for each orientation).

Your `task15()` function in the `analysis.py` module should return the spot plot (matplotlib figure object) and focal point for the plano-convex system followed by the spot plot and focal point of the convex-plano system as a 4 element tuple

Now let us investigate how each of these orientations compares to the diffraction scale by plotting (as we did before) the input beam radius against RMS/diffraction scale at their respective focal points.

Task 16

For the same range of input beam radii (0.1mm to 10mm), propagate your bundle through each lens orientation, ending at an OutputPlane (located at the focal point). Calculate the RMS value at the focal point for each orientation of the lens.

Plot the radial dependence of the RMS value for each orientation, along with the diffraction scale from above (again at wavelength = 588nm) on the same figure.

Your `task16()` function should return this matplotlib figure object, along with the RMS value (for both orientations) and diffraction scale value for input beam radius of 3.5mm

Optimising a BiConvex lens

Hopefully from `Task 16` above you found that your PlanoConvex lens with the curved side first produced a smaller RMS value (at the focal point) for all input beam radii. We will now see if we can get a smaller RMS spot size at the same focal point, using a bi-convex lens.

Task 17.

Create a new class `BiConvex` in your `lenses.py` module. This should be instantiable with the same arguments as the `PlanoConvex` class. Thus you should be able to initialise it as below:

```
>>> biconvex_lens = BiConvex(z_0=100., curvature1=0.02, curvature2=-0.02,
n_inside=1.5168, n_outside=1., thickness=5., aperture=50.)
```

We will be comparing our `BiConvex` lens to the `PlanoConvex` with the curved side first that you used in `Task 16`, namely:

```
>>> cpl = PlanoConvex(z_0=100., curvature1=0.02, curvature2=0., n_inside=1.5168,
n_outside=1., thickness=5., aperture=50.)
```

Trace a `RayBundle` through the `PlanoConvex` lens to an `OutputPlane` placed at the focal point of this lens and note the RMS spot size at this point. We will now attempt to optimise the curvature of a `BiConvex` lens to minimise the RMS at the same focal point.

To do this you will need to use the `SciPy minimise` function allowing the curvatures `c1` and `c2` to float and be parameters to this minimisation.

With this done you should combine the spot plots of both the `PlanoConvex` and `BiConvex` lenses on the same figure. Ensure that your `task17()` function in `analysis.py` module returns the combined spot plot as well as the RMS for the `PlanoConvex` and `BiConvex`

lenses at the focal point for the plano-convex lens.

Reflection

A simple addition to our simulation would be to introduce the modelling of reflective surfaces. We shall now attempt to add a new spherical reflecting surface and test that it is working properly.

Task 18.

Add a new class `SphericalReflection` to your `elements.py` module. This should be instantiable as below

```
>>> sph_refl = SphericalReflection(z_0=100., aperture=6., curvature=-0.02)
```

Instantiate the reflecting surface with the parameters outlined above and ensure that you implement any additional methods necessary for your simulation to work correctly.

In your `task18()` function in `analysis.py`, trace a standard `RayBundle` with $r_{max} = 5mm$ to the reflecting surface followed by an `OutputPlane` placed at $50mm$.

Ensure that the `task18()` function returns both this trace plot and the focal point of your reflecting surface.

Quantifying the spherical aberration

By this stage we are aware that the PlanoConvex lens displays signs of spherical aberration but we would like to study it more rigorously. We would like to try and quantify the amounts of *both* transverse as well as longitudinal spherical aberration.

Task 19.

To start with let's consider the transverse spherical aberration. In the `task19` function of your `analysis.py` module, we will create a `PlanoConvex` lens with our usual parameters, namely:

```
>>> ConvexPlano(z_0=100., aperture=50., curvature=0.02, n_inside=1.5168,
n_outside=1., thickness=5.)
```

Using this lens, and propagating a ray bundle to an `OutputPlane` located at the focal point, we shall investigate firstly how the amount of transverse spherical aberration depends on input beam size for a fixed sized lens. We shall do this by plotting the RMS

spot size (at the focal point) as a function of input beam radius. This should be fairly trivial as we have already done this in `Task 16`. The only change we shall make is to widen the range of input radii from (0.1mm to 20mm).

We shall now investigate the longitudinal spherical aberration. To make our lives easier, we will start by defining a helper method within your `Ray` class. This method should be called `z_int` and should return the location that the given ray will intercept with the z (optical) axis. If, given the rays current direction, it will **not** intercept the optical axis then the method should raise an exception.

```
class Ray
```

```
...
```

```
    z_int(self)
```

This method calculates and returns the z intercept of the ray, given its current direction.

Now that we have this helper method, you should be able to plot the z intercept of your rays within a bundle as a function of their transverse distance from the optical axis. Make sure to consider rays over the same range of transverse distances as above, i.e. between (0.1mm and 20mm). To finish this plot off, place a horizontal line at the focal point.

Ensure that the `task19()` function returns both the transverse and longitudinal spherical aberration plots

Dispersive materials

A dispersive material is a medium where the phase velocity of a wave depends on its frequency, causing different components of a wave to travel at different speeds. In optics, this means the refractive index varies with wavelength, separating light into its spectral colors

We shall extend our code to allow us to simulate dispersion effects.

Task 20.

Firstly we will need to modify our `Ray` class to allow us to set and store a wavelength. We will add an additional parameter:

- **wavelength** - the wavelength of the ray

This should be defaulted to a wavelength of 588 nm and we should be careful to not

break the existing behaviour. This means that you should be able to instantiate your Ray objects in any of the ways below.

```
>>> ray1 = Ray()
>>> ray2 = Ray(pos=[0, 0, 0], direc=[0, 0, 1])
>>> ray3 = Ray(pos=[0, 0, 0], direc=[0, 0, 1], wavelength=588e-9)
```

To access this new internal variable we will need a new accessor method as detailed below:

```
class Ray
```

```
...
```

```
    wavelength(self)
```

This method returns the `Ray` object's wavelength.

Next we shall create a new class `DispersiveMaterial` in your `physics.py` module. It will contain the Sellmeier **B** and **C** coefficients needed to calculate the refractive index at a given wavelength. You should be able to construct your `DispersiveMaterial` objects as below:

```
>>> material = DispersiveMaterial(b_coeffs=(1., 2., 3.), c_coeffs=(4., 5., 6.))
```

Your `DispersiveMaterial` class should have a single method to calculate the refractive index for a given wavelength using the stored Sellmeier coefficients as defined below.

```
class DispersiveMaterial
```

```
    ref_index(self, wavelength)
```

This method should return the refractive index calculated for light with given wavelength.

Your `SphericalRefraction` class (as well as your lenses) should be able to take either a float or a `DispersiveMaterial` object for the `n_1` and `n_2` parameters e.g.

```
>>> material = DispersiveMaterial(b_coeffs=(1., 2., 3.), c_coeffs=(4., 5., 6.))
>>> sr = SphericalRefraction(z_0=10, aperture=5, curvature=0.2, n_1=1.,
n_2=material)
```

Your classes will probably already be able to do this as they will likely, in the `__init__` method, simply take this parameter and store it.

You will however now need to modify the `n_1` and `n_2` methods in your

`SphericalRefraction` class to be able to take a wavelength as a parameter as below:

Note: they can have a sensible default value to preserve the way you have been calling them up until now, I show both ways below but you should pick one and stick with it

class SphericalRefraction

...

n_1(self, wavelength)

return the refractive index on the left ($z < z_0$) of the surface

n_2(self, wavelength=None)

return the refractive index on the right ($z > z_0$) of the surface

These methods should return simply the numerical value of the stored `n_1` or `n_2` if the `SphericalRefraction` object was created with a simple float value. Alternatively they should calculate the refractive index by calling the `ref_index` method of the underlying `DispersiveMaterial` object that was stored in `n_1` or `n_2` using the wavelength parameter.

Finally you will need to change any places in your code where your `n_1` or `n_2` methods are called to make sure you are now passing the `Ray` objects wavelength if needed. This includes places like `SphericalRefraction.propagate_ray()` and `SphericalRefraction.focal_point()`.

Note: there could be others depending on the layout of your code

Now let's use our new extension to model the dispersion of light through borosilicate crown glass BK7 (specifically SCHOTT N-BK7). In the `task20` function in your `analysis.py` module, create a `BiConvex` lens made out of BK7 and trace several rays through it. Each of the rays should start from the same spot with the same direction. The only difference is that you should choose different wavelengths for them. It is also useful if you colour them to match these wavelengths.

Propagate the rays through your lens to an `OutputPlane` and check that you can see the different rays splitting.

Ensure that the `task20()` function returns the ray track plot.

Warning

Please ensure that all code as well as all plots are committed and pushed to GitHub.

Failure to do so will result in you not having the correct mark assigned.

MAKE SURE THEY ARE COMMITTED AND PUSHED!

You have now completed the main tasks of this project. Now is the time to go back through your code and improve it, as discussed in the worksheets. You should consider your naming conventions, docstrings, and class inheritance trees. Remember to check your changes have not made any tests now fail.



Stop

At this point if you have finished everything talk to a demonstrator about possible extensions.